

I- 1ère partie : Déployer une base de données.

1- Créer un espace de nom

Création impérative du namespace : `kubectl create namespace projet-3k8s`

Suppression du namespace : `kubectl delete namespace projet-3k8s`

On utilise la méthode impérative pour créer immédiatement le namespace sans nécessiter de fichier YAML.

Cela permet une mise en place rapide, mais n'est pas idéal pour la gestion des configurations sur le long terme.

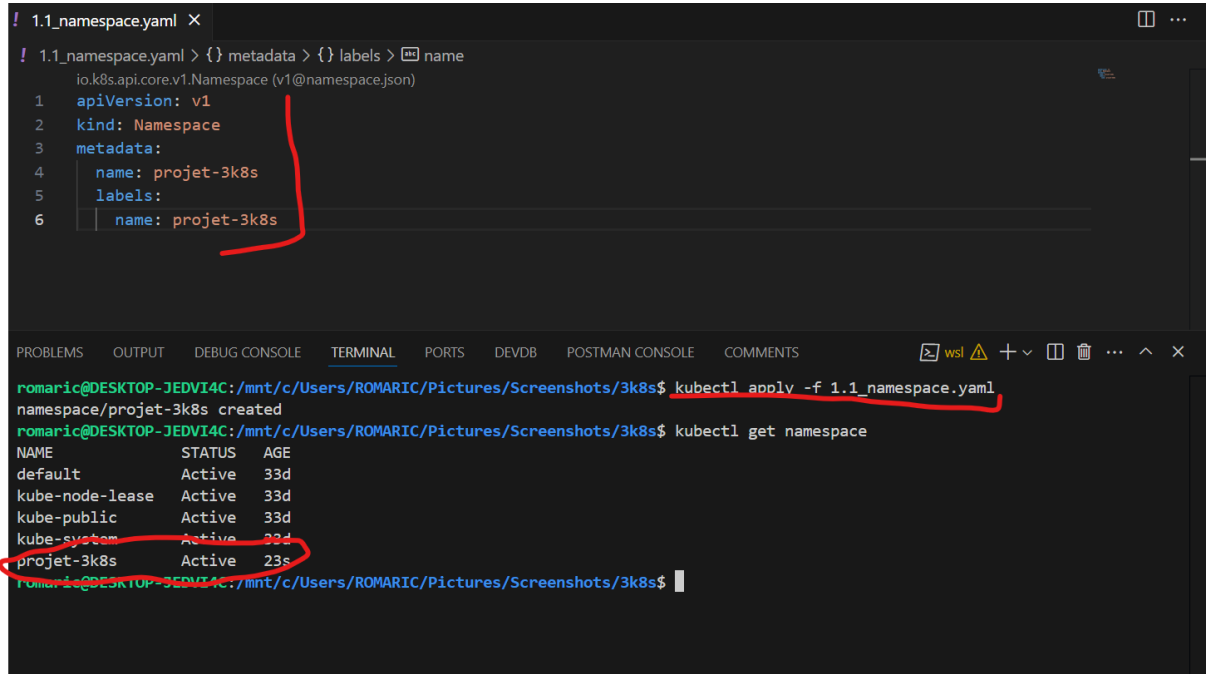
```
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC$ kubectl get namespace
NAME                STATUS    AGE
default             Active    33d
kube-node-lease     Active    33d
kube-public         Active    33d
kube-system         Active    33d
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC$ kubectl create namespace projet-3k8s
namespace/projet-3k8s created
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC$ kubectl get namespace
NAME                STATUS    AGE
default             Active    33d
kube-node-lease     Active    33d
kube-public         Active    33d
kube-system         Active    33d
projet-3k8s        Active    9s
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC$ kubectl delete namespace projet-3k8s
namespace "projet-3k8s" deleted
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC$ kubectl get namespace
NAME                STATUS    AGE
default             Active    33d
kube-node-lease     Active    33d
kube-public         Active    33d
kube-system         Active    33d
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC$ |
```

?

Création déclarative du namespace : `kubectl apply -f 1.1_namespace.yaml`

On crée un fichier YAML (namespace.yaml) contenant la définition du namespace.

On applique ce fichier avec `kubectl apply -f namespace.yaml`, ce qui permet une meilleure traçabilité et versioning du namespace dans un projet Kubernetes.

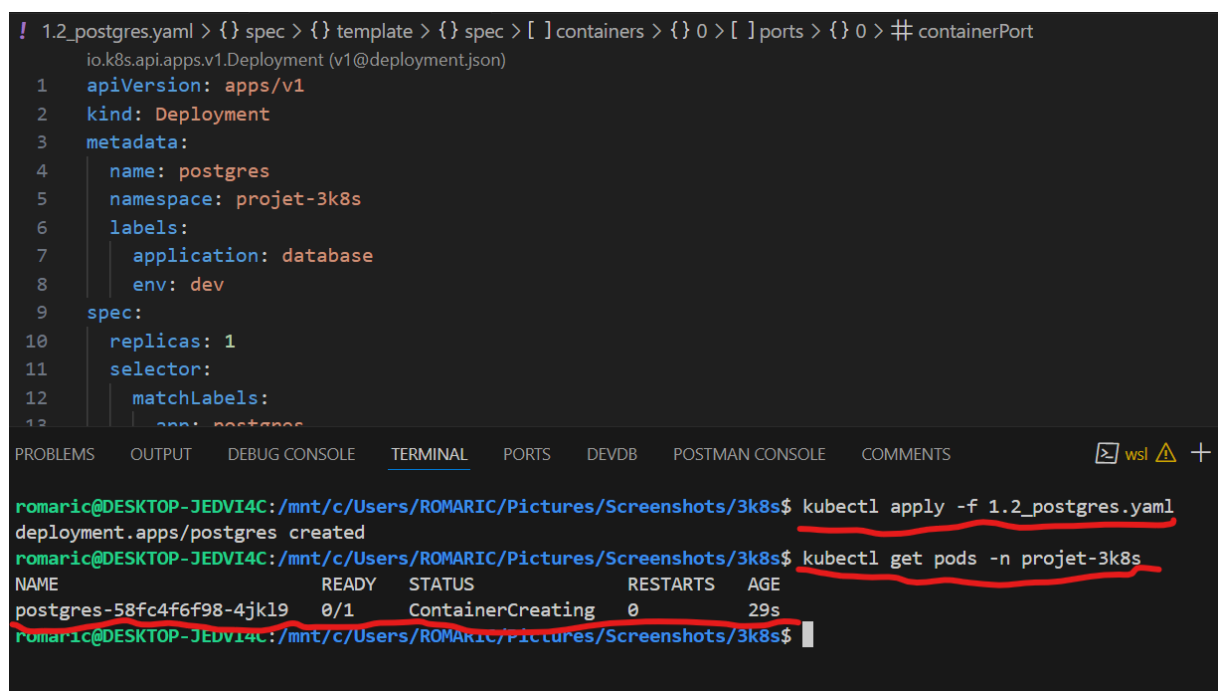


```
! 1.1_namespace.yaml X
! 1.1_namespace.yaml > {} metadata > {} labels > name
io.k8s.api.core.v1.Namespace (v1@namespace.json)
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: projet-3k8s
5    labels:
6      name: projet-3k8s

romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl apply -f 1.1_namespace.yaml
namespace/projet-3k8s created
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get namespace
NAME              STATUS    AGE
default           Active    33d
kube-node-lease   Active    33d
kube-public       Active    33d
kube-system       Active    33d
projet-3k8s       Active    23s
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$
```

2- Créer un déploiement

Commande de déploiement : `kubectl apply -f postgres-deployment.yaml`



```
! 1.2_postgres.yaml > {} spec > {} template > {} spec > [ ] containers > {} 0 > [ ] ports > {} 0 > # containerPort
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: postgres
5    namespace: projet-3k8s
6    labels:
7      application: database
8      env: dev
9  spec:
10   replicas: 1
11   selector:
12     matchLabels:
13       application: postgres

romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl apply -f 1.2_postgres.yaml
deployment.apps/postgres created
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get pods -n projet-3k8s
NAME                                READY   STATUS    RESTARTS   AGE
postgres-58fc4f6f98-4jk19          0/1     ContainerCreating   0          29s
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$
```

Pour afficher tous les objets Kubernetes explicitement ou implicitement créés dans le namespace projet-3k8s : `kubectl get all -n projet-3k8s`

Cela affichera :

- Les pods
- Les services
- Les déploiements
- Les replicasets

Afficher les détails d'un objet spécifique : `kubectl describe deployment postgres -n projet-3k8s`

```
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get all -n projet-3k8s
NAME                                READY   STATUS    RESTARTS   AGE
pod/postgres-58fc4f6f98-4jkl9      1/1     Running   0           43m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/postgres            1/1     1             1           43m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/postgres-58fc4f6f98 1         1         1       43m
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl describe deployment postgres -n projet-3k8s
Name:                                postgres
Namespace:                          projet-3k8s
CreationTimestamp:                   Sun, 23 Mar 2025 21:14:40 +0100
Labels:                              application=database
                                      env=dev
Annotations:                         deployment.kubernetes.io/revision: 1
Selector:                            app=postgres
Replicas:                            1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType:                        RollingUpdate
MinReadySeconds:                     0
RollingUpdateStrategy:               25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=postgres
  Containers:
    postgres:
      Image:          kubway/persistent-flask-app-db:1.0.0
      Port:           5432/TCP
      Host Port:      0/TCP
      Limits:
        cpu:          500m
        memory:        1Gi
      Requests:
        cpu:           250m
```

Vérifier sur quel nœud est exécuté le pod: `kubectl get pods -n projet-3k8s -o wide`

Afficher la configuration YAML actuelle d'un objet : `kubectl get deployment postgres -n projet-3k8s -o yaml`

```
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get pods -n projet-3k8s -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE             NOMINATED NODE   READINESS GATES
postgres-58fc4f6f98-4jkl9          1/1     Running   0           4h9m  10.1.0.22     docker-desktop    <none>           <none>
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get deployment postgres -n projet-3k8s -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{},"labels":{"application":"database","env":"dev"},"name":"postgres","namespace":"projet-3k8s"},"spec":{"replicas":1,"selector":{"matchLabels":{"app":"postgres"}},"template":{"meta
data":{"labels":{"app":"postgres"},"spec":{"containers":[{"env":[{"name":"POSTGRES_DB","value":"postgre-database"},{"name":"POS
GRES_USER","value":"user"},{"name":"POSTGRES_PASSWORD","value":"P@ssw0rd"}],"image":"kubway/persistent-flask-app-db:1.0.0","nam
e":"postgres","ports":[{"containerPort":5432}], "resources":{"limits":{"cpu":"500m","memory":"1Gi"},"requests":{"cpu":"250m","mem
ory":"512Mi"}}}}}}}}
  creationTimestamp: "2025-03-23T20:14:40Z"
  generation: 1
  labels:
    application: database
    env: dev
    name: postgres
  namespace: projet-3k8s
  resourceVersion: "200894"
  uid: cbc43a08-1bc6-4774-9326-c984c0fae5dc
spec:
  progressDeadlineSeconds: 600
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: postgres
  strategy:
```

Vérifier l'API et la version d'un objet Kubernetes (deployment) : `kubectl api-resources | grep deployment`

```
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl api-resources | grep deployment
deployments          deploy      apps/v1      true      Deployment
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$
```

Liste de quelques commandes effectuées

Objectif	Commande
Créer un namespace	<code>kubectl create namespace projet-3k8s</code>
Supprimer un namespace	<code>kubectl delete namespace projet-3k8s</code>
Appliquer un fichier YAML	<code>kubectl apply -f <fichier.yaml></code>
Lister tous les objets	<code>kubectl get all -n projet-3k8s</code>
Voir les détails d'un objet	<code>kubectl describe <type> <nom> -n projet-3k8s</code>
Voir les logs d'un conteneur	<code>kubectl logs postgres-58fc4f6f98-4jkl9 -n projet-3k8s</code>
Voir les événements	<code>kubectl get events -n projet-3k8s</code>
Suivre l'évolution d'un objet	<code>kubectl get pods -n projet-3k8s --watch</code>

Objectif	Commande
Voir sur quel nœud tourne un pod	<code>kubectl get pods -o wide -n projet-3k8s</code>
Exporter un objet en YAML	<code>kubectl get deployment postgres -n projet-3k8s -o yaml > 1.2_postgres.yaml</code>

3- Créer et utiliser un secret

Commande de création d'un secret : `kubectl apply -f 1.3_postgres-secret.yaml`

- ◆ **data** → Les valeurs doivent être en base64 (Kubernetes les décode au moment de l'utilisation).
- ◆ **stringData** → Les valeurs sont en texte brut, Kubernetes se charge de les encoder en base64

Donc pour plus de sécurité, nous avons opté pour 'data'

```
! 1.3_postgres-secret.yaml > {} data
  io.k8s.api.core.v1.Secret (v1@secret.json)
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: postgres-secret
5    namespace: projet-3k8s
6  type: Opaque
7  data:
8    POSTGRES_DB: cG9zdGdyZS1kYXRhYmFzZQ==
9    POSTGRES_USER: dXN1cg==
10   POSTGRES_PASSWORD: UEAzc3cwcmQ=
11

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  DEVDB  POSTMAN CONSOLE  COMMENTS

romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl apply -f 1.3_postgres-secret.yaml
secret/postgres-secret created
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get secrets -n projet-3k8s
NAME          TYPE      DATA      AGE
postgres-secret  Opaque    3          60s
```

On **remplace** les valeurs en dur des variables d'environnement (env) par des références à un **Secret Kubernetes**.

valueFrom.secretKeyRef permet de **récupérer** les valeurs stockées dans postgres-secret et de les injecter dans le conteneur.

```

9 spec:
14   template:
18     spec:
19       containers:
20         - name: postgres
24           valueFrom:
25             secretKeyRef:
26               name: postgres-secret
27               key: POSTGRES_DB
28         - name: POSTGRES_USER
29           valueFrom:
30             secretKeyRef:
31               name: postgres-secret
32               key: POSTGRES_USER
33         - name: POSTGRES_PASSWORD
34           valueFrom:
35             secretKeyRef:
36               name: postgres-secret
37               key: POSTGRES_PASSWORD
38       resources:
39         requests:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS DEVDB POSTMAN CONSOLE COMMENTS

romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s\$ kubectl apply -f 1.3_postgres.yaml

deployment.apps/postgres configured

4- Créer un service interne au cluster

Créer le service : `kubectl apply -f 1.4_postgres-svc.yaml`

Voir les détails du service : `kubectl describe svc postgres-svc -n projet-3k8s`

- Le selector: `app: postgres` assure que le service trouve et expose uniquement les pods PostgreSQL déployés avec l'étiquette `app=postgres`.
- Pas besoin d'accès externe, donc **ClusterIP** est le type.
- Il est **non exposé à l'extérieur du cluster**.
- port: 5432 → Ce port est celui utilisé pour les connexions à la base de données PostgreSQL dans le cluster.
- targetPort: 5432 → Il correspond au port du conteneur PostgreSQL

```
! 1.4_postgres-svc.yaml > {} spec
  io.k8s.api.core.v1.Service (v1@service.json)
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: postgres-svc
5    namespace: projet-3k8s
6  spec:
7    selector:
8      app: postgres

romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl describe svc postgres-svc -n projet-3k8s
Name:                postgres-svc
Namespace:           projet-3k8s
Labels:              <none>
Annotations:         <none>
Selector:            app=postgres
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  None
IPs:                 None
Port:                <unset> 5432/TCP
TargetPort:          5432/TCP
Endpoints:           10.1.0.34:5432
Session Affinity:    None
Internal Traffic Policy: Cluster
Events:              <none>
```

Lister les endpoints des pods associés ; `kubectl get endpoints -n projet-3k8s`

```
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get endpoints -n projet-3k8s
NAME           ENDPOINTS          AGE
postgres-svc  10.1.0.34:5432    4m19s
```

5- Créer un PersistentVolumeClaim

Créer le PVC avec la commande : `kubectl apply -f 1.5_postgres-pvc.yaml`

- `name: postgres-pvc` → Nom du PVC.
- `namespace: projet-3k8s` → Correspond au namespace défini précédemment.
- `ReadWriteOnce` → Le volume peut être monté en lecture/écriture par un seul nœud à la fois.
- Adapté aux bases de données comme PostgreSQL.
- `storage: 1Gi` → Alloue 1 Go de stockage pour PostgreSQL.
- `storageClassName: standard` → Permet d'utiliser un provisioner automatique pour créer un PersistentVolume (PV) correspondant.

```
! 1.5_postgres-pvc.yaml > {} spec
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: postgres-pvc
5    namespace: projet-3k8s
6  spec:
7    accessModes:
8      - ReadWriteOnce # Accès en lecture/écriture par un seul nœud
9    resources:
10     requests:
11       storage: 1Gi # Demande 1Gi de stockage
12     persistentVolumeClaimName: standard # Utiliser le provisioner pour créer le stockage

romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl apply -f 1.5_postgres-pvc.yaml
persistentvolumeclaim/postgres-pvc created
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl apply -f 1.5_postgres.yaml
deployment.apps/postgres configured
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$
```

```
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get all,pvc,pv,secret -n projet-3k8s
NAME                                READY   STATUS    RESTARTS   AGE
pod/postgres-65f45465d5-kflbz       1/1     Running   1 (13h ago)  2d15h
pod/postgres-b7dcb57ff-mmrrp        0/1     Pending   0           114s

NAME                                TYPE     CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/postgres-svc                ClusterIP  None          <none>         5432/TCP    29m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/postgres             1/1     1            1           4d6h
replicaset.apps/postgres-58fc4f6f98  0       0            0           4d6h
replicaset.apps/postgres-65f45465d5  1       1            1           2d15h
replicaset.apps/postgres-b7dcb57ff   1       1            0           114s

NAME                                STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/postgres-pvc  Pending            2Gi        RWO         Retain         Bound         default/mysql-data      manuel             <unset>
                                     37d

NAME                                CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                STORAGECLASS   VOLUMEATTRIBUTE
SCLASS REASON AGE
persistentvolume/vol-1  2Gi        RWO         Retain         Bound   default/mysql-data   manuel             <unset>
                                     37d

NAME                                TYPE    DATA   AGE
secret/postgres-secret              Opaque  3       2d16h
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$
```


II- 2ème partie : Déployer une application

1- Créer un déploiement

Connexion à PostgreSQL via le Secret :

- **Nom de la base (POSTGRES_DB)** → récupéré depuis postgres-secret
- **Utilisateur (POSTGRES_USER)** → récupéré depuis postgres-secret
- **Mot de passe (POSTGRES_PASSWORD)** → récupéré depuis postgres-secret
- **Hôte PostgreSQL (POSTGRES_HOST)** → défini comme "postgres-svc" (le **nom du service PostgreSQL**).

Création du deployment : `kubectl apply -f 2.1_flask-app.yaml`

Vérifier le déploiement : `kubectl get deployments -n projet-3k8s`

Vérifier les pods : `kubectl get pods -n projet-3k8s`

Liste des pods avec le label `app=flask-app` : `kubectl get pods --all-namespaces -l app=flask-app`.

```
! 2.1_flask-app.yaml > {} spec > {} template > {} spec > [ ] containers > {} 0 > {} resources > {} limits
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: flask-app
5    namespace: projet-3k8s
6    labels:

romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl apply -f 2.1_flask-app.yaml
deployment.apps/flask-app created
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get deployments -n projet-3k8s
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
flask-app  0/3     3            0           40s
postgres  1/1     1            1           5d15h
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get pods -n projet-3k8s
NAME                                READY   STATUS              RESTARTS   AGE
flask-app-b74b96c48-6cd9m            0/1     ContainerCreating   0           65s
flask-app-b74b96c48-hb25b            0/1     ContainerCreating   0           65s
flask-app-b74b96c48-hnlsc            0/1     ContainerCreating   0           65s
postgres-65f45465d5-kflbz           1/1     Running             2 (12h ago) 4d
postgres-b7dc57ff-mmrrp              0/1     Pending             0           33h
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get pods --all-namespaces -l app=flask-app
NAMESPACE   NAME                                READY   STATUS              RESTARTS   AGE
projet-3k8s  flask-app-b74b96c48-6cd9m            1/1     Running            0           100s
projet-3k8s  flask-app-b74b96c48-hb25b            1/1     Running            0           100s
projet-3k8s  flask-app-b74b96c48-hnlsc            1/1     Running            0           100s
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$
```

2- Créer un service

Sélecteur (selector) : Cible tous les pods qui ont `app=flask-app` (correspond au déploiement `flask-app`).

Type de service (type: NodePort) : Permet d'accéder à l'application depuis l'extérieur

- `port: 80` → Port exposé par le service.
- `targetPort: 5000` → Port du conteneur où Flask tourne.
- `nodePort: 30080` → Port sur lequel le service est accessible depuis l'extérieur.

Appliquer le fichier YAML : `kubectl apply -f 2.2_flask-app-svc.yaml`

Vérifier le service : `kubectl get svc -n projet-3k8s`

```
! 2.2_flask-app-svc.yaml > {} spec > [ ] ports > {} 0
  io.k8s.api.core.v1.Service (v1@service.json)
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: flask-app-svc
5    namespace: projet-3k8s
6  spec:
7    selector:
8      app: flask-app
9    type: NodePort # Permet d'exposer le service à l'extérieur du cluster
10   ports:

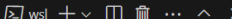
romaric@DESKTOP-JEDVI4C: /mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl apply -f 2.2_flask-app-svc.yaml
service/flask-app-svc created
romaric@DESKTOP-JEDVI4C: /mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get svc -n projet-3k8s
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
flask-app-svc NodePort    10.106.79.166 <none>        80:30080/TCP     17s
postgres-svc  ClusterIP   None         <none>        5432/TCP         34h
```

3- Créer un ingress

Créer l'Ingress dans Kubernetes : `kubectl apply -f flask-app-ingress.yaml`

Vérifier que l'Ingress est bien créé : `kubectl get ingress -n projet-3k8s`

```
1 1.5_postgres-pv.yaml > {} spec > {} hostPath
2 apiVersion: v1
3 kind: PersistentVolume
4 metadata:
5   name: postgres-pv
6   namespace: projet-3k8s
7 spec:
8   capacity:
9     storage: 1Gi
10   volumeMode: Filesystem
11   accessModes:
12     - ReadWriteOnce
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS DEVDB POSTMAN CONSOLE COMMENTS 

```
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl apply -f 1.5_postgres-pv.yaml
persistentvolume/postgres-pv created
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$ kubectl get pvc -n projet-3k8s
kubectl get pv
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
postgres-pvc	Bound	postgres-pv	1Gi	RWO	standard	<unset>	34h

```
NAME CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM STORAGECLASS VOLUMEATTRIBUTESCLASS
```

REASON	AGE	NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS
postgres-pv	16s	1Gi	RWO	Retain	Bound	projet-3k8s/postgres-pvc	standard	<unset>	
vol-1	38d	2Gi	RWO	Retain	Bound	default/mysql-data	manuel	<unset>	

```
romaric@DESKTOP-JEDVI4C:/mnt/c/Users/ROMARIC/Pictures/Screenshots/3k8s$
```

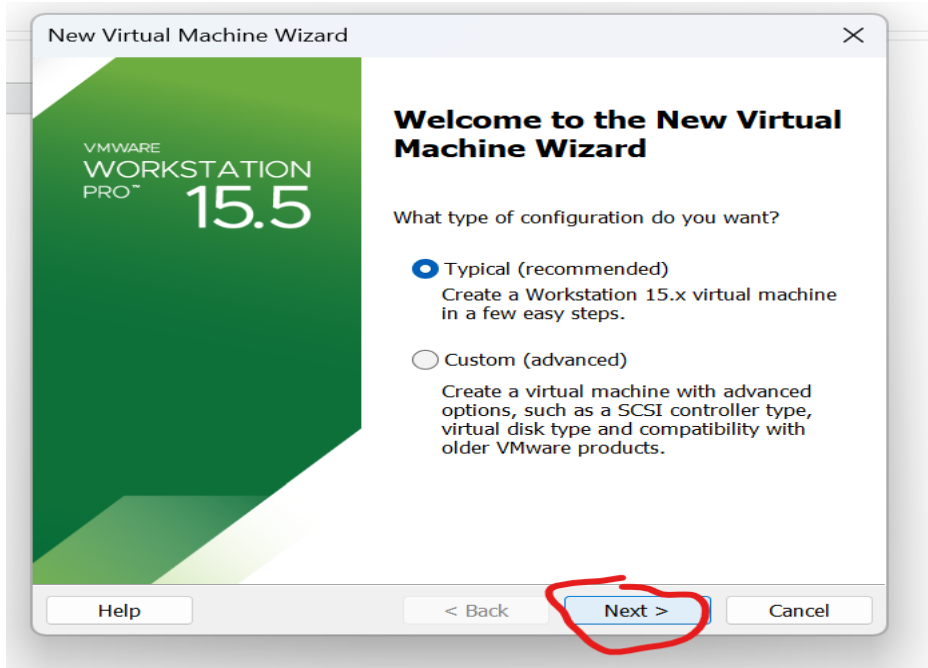
4- Tester l'application

III- Bonus

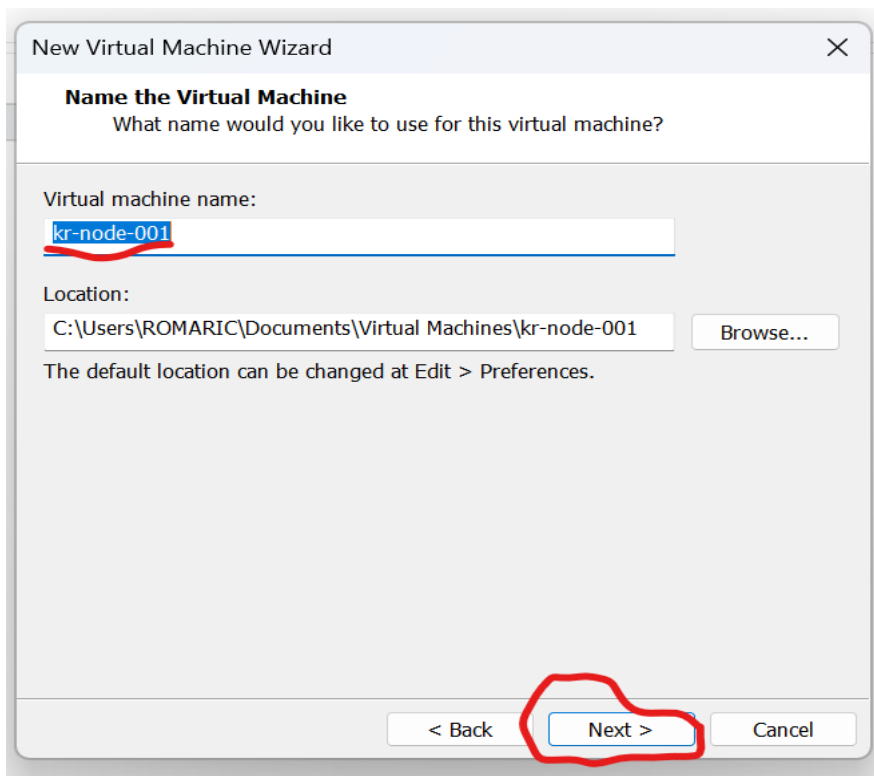
1- 1ère partie : Créer 3 serveurs virtuels

Nous avons choisi VMWARE pour plus de stabilité au niveau du réseau interne et externe des machines virtuelles.

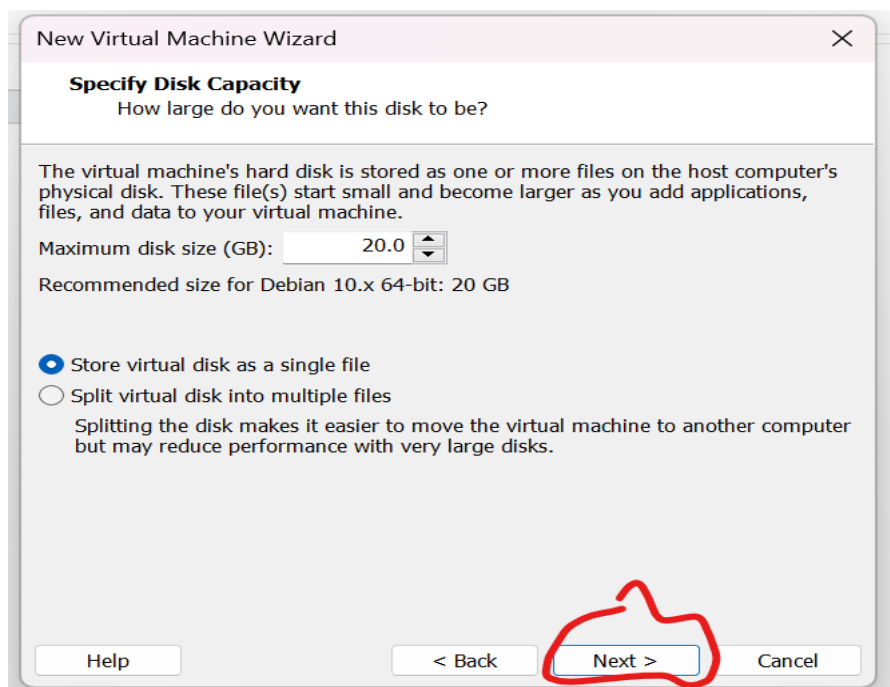
Création d'une nouvelle machine virtuelle (ces étapes de création sont valables pour les trois machines virtuelles).



ajout du nom de la machine virtuelle



Mettre un taille maximale de 2Go



New Virtual Machine Wizard

Specify Disk Capacity
How large do you want this disk to be?

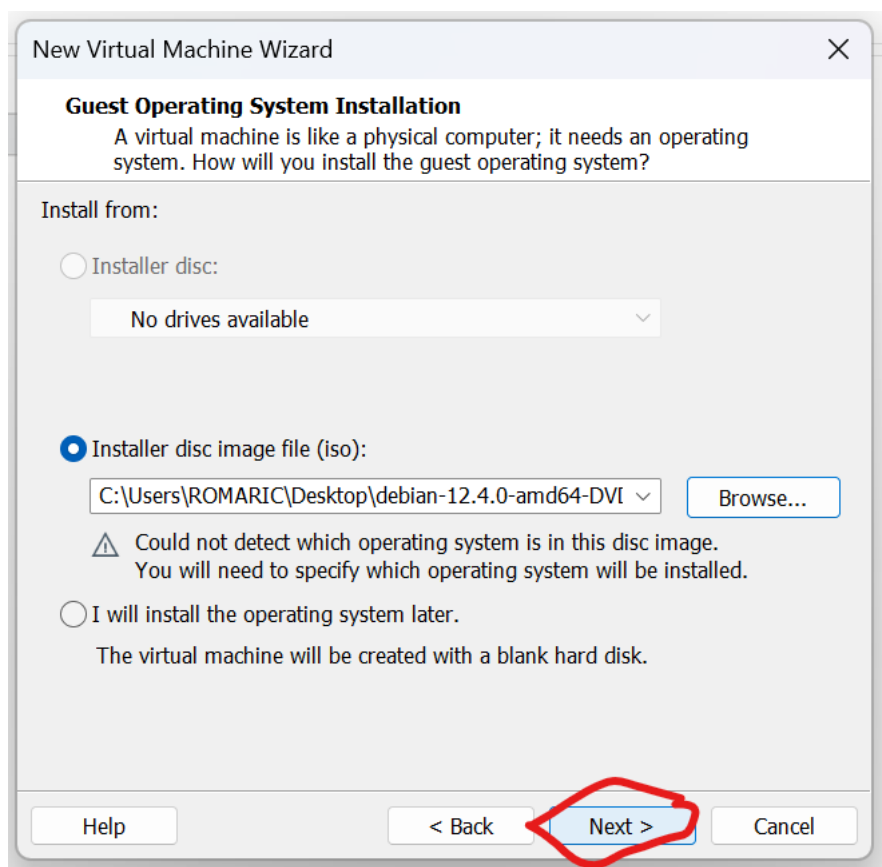
The virtual machine's hard disk is stored as one or more files on the host computer's physical disk. These file(s) start small and become larger as you add applications, files, and data to your virtual machine.

Maximum disk size (GB): 

Recommended size for Debian 10.x 64-bit: 20 GB

☒ Store virtual disk as a single file
☐ Split virtual disk into multiple files
Splitting the disk makes it easier to move the virtual machine to another computer but may reduce performance with very large disks.

Help < Back **Next >** Cancel



New Virtual Machine Wizard

Guest Operating System Installation
A virtual machine is like a physical computer; it needs an operating system. How will you install the guest operating system?

Install from:

☐ Installer disc:
 

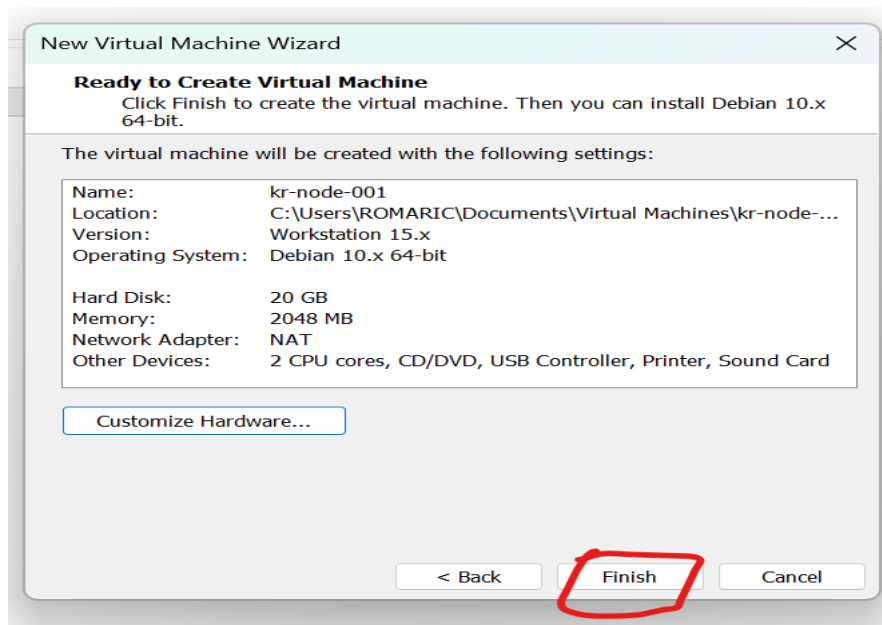
☒ Installer disc image file (iso):
 

 Could not detect which operating system is in this disc image.
You will need to specify which operating system will be installed.

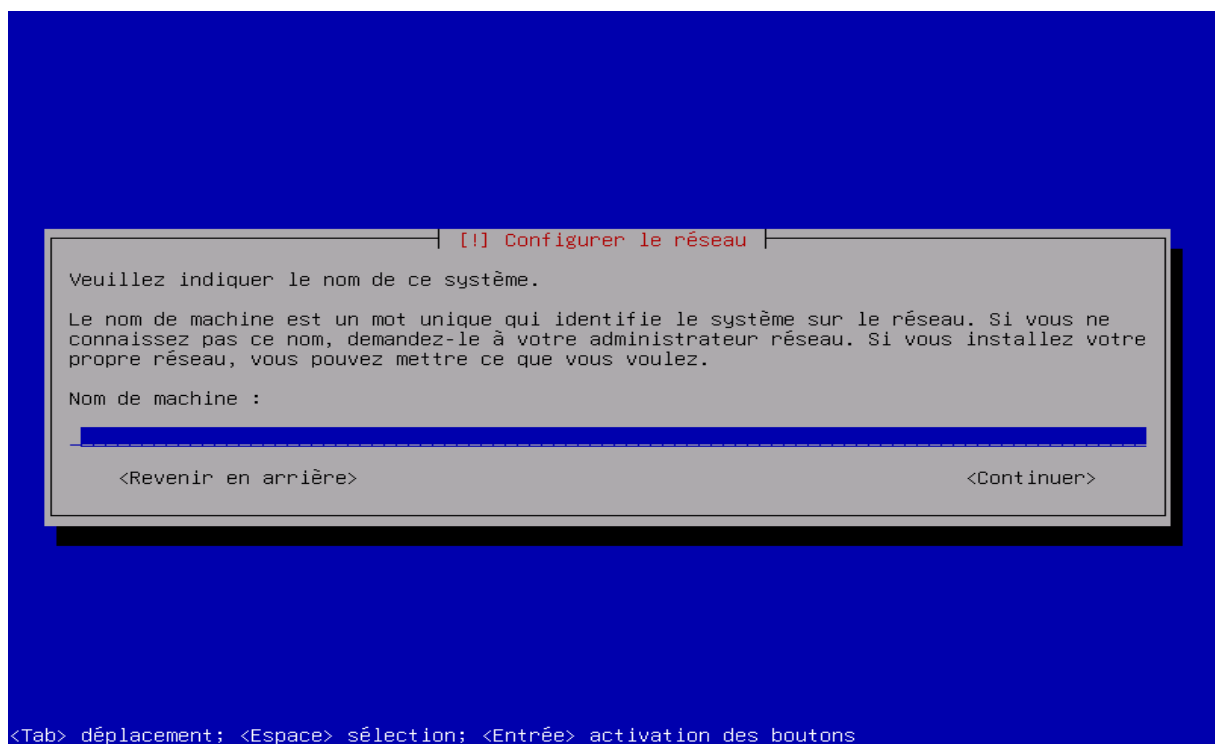
☐ I will install the operating system later.
The virtual machine will be created with a blank hard disk.

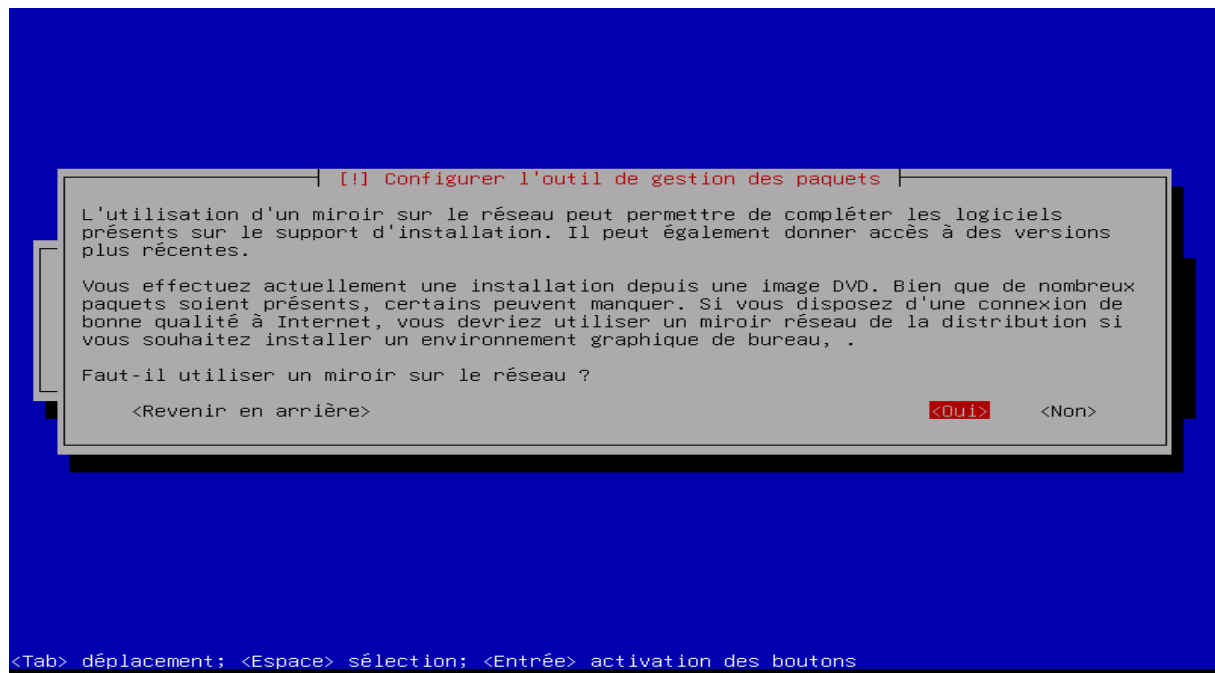
Help < Back **Next >** Cancel

On clique ensuite sur finish et lorsque le machine se démarre, on choisit le mode d'installation 'install'.

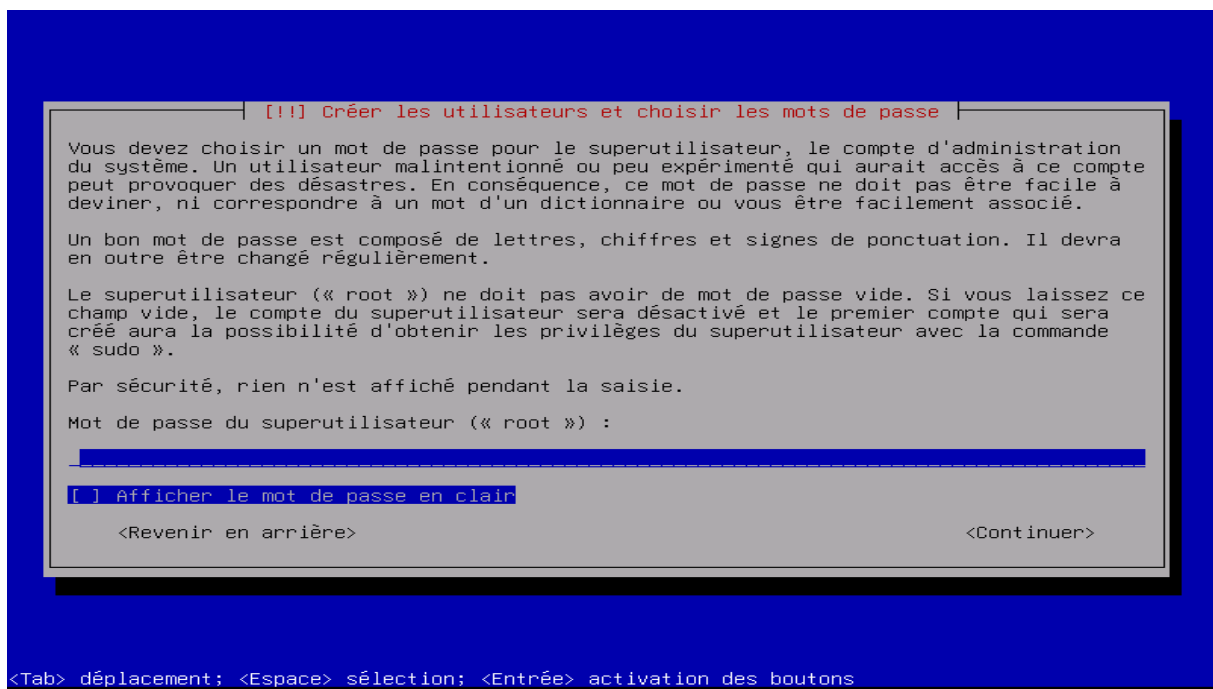


Mettre le nom de la machine pour l'identifier sur le réseau

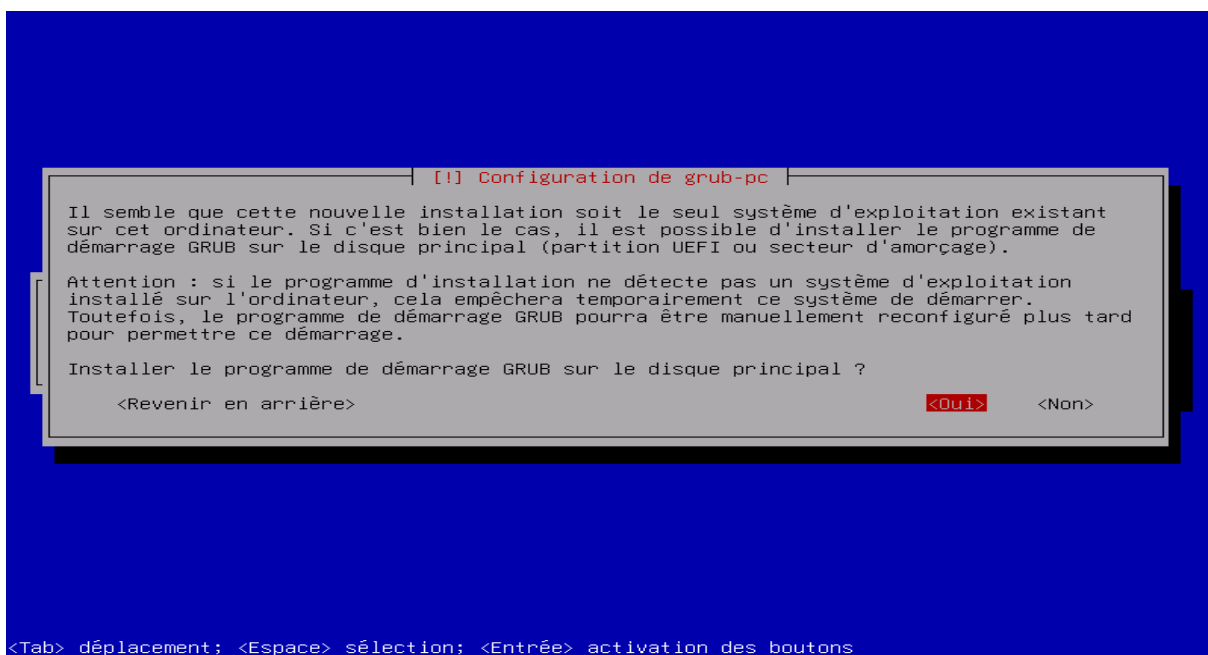
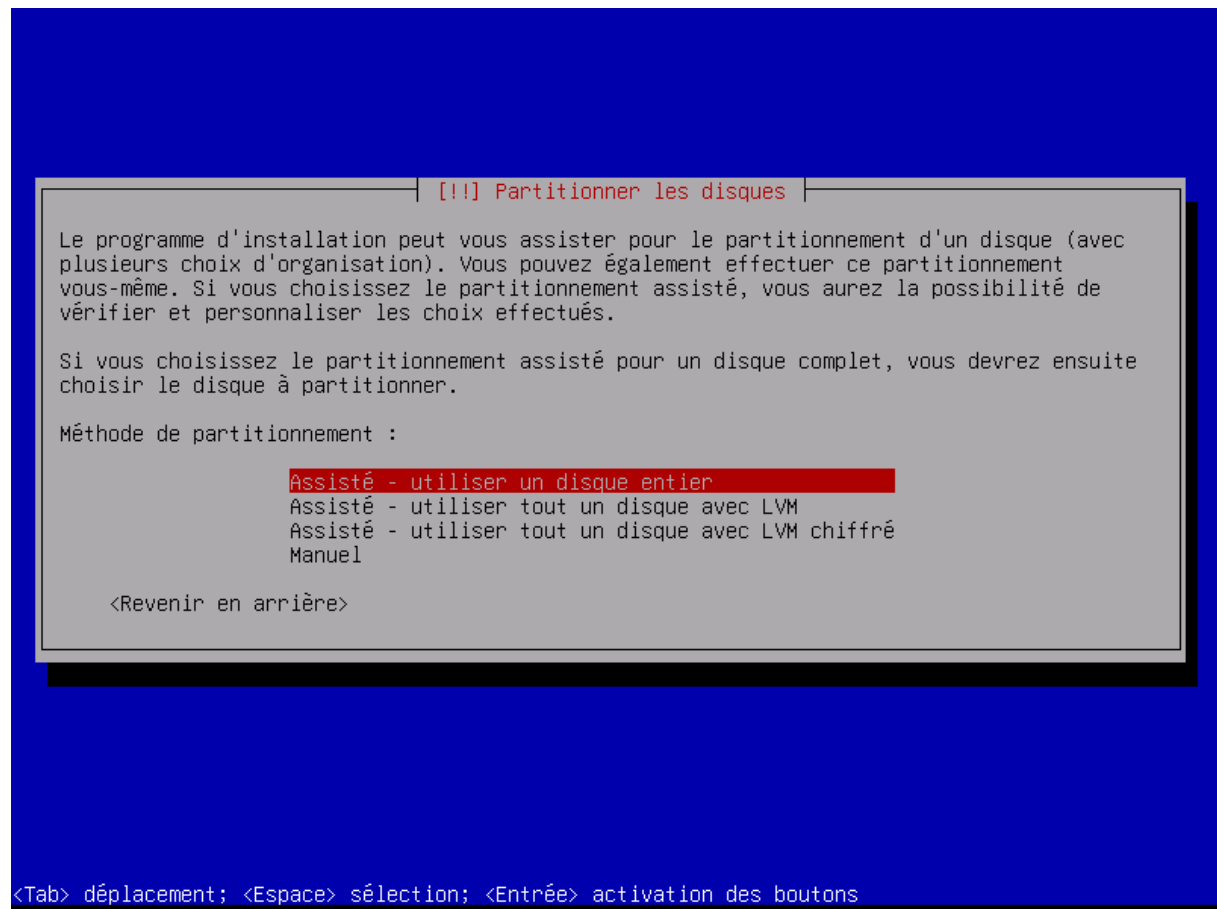


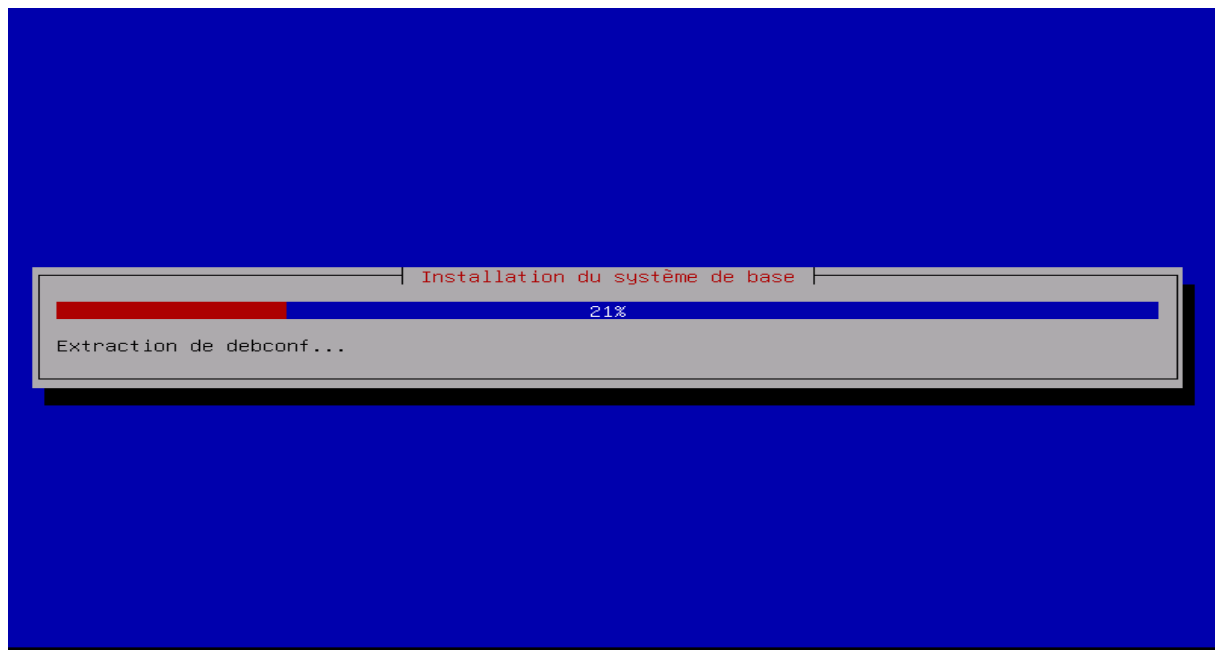


Mettre le mot de passe de l'administrateur et ensuite définir un utilisateur.

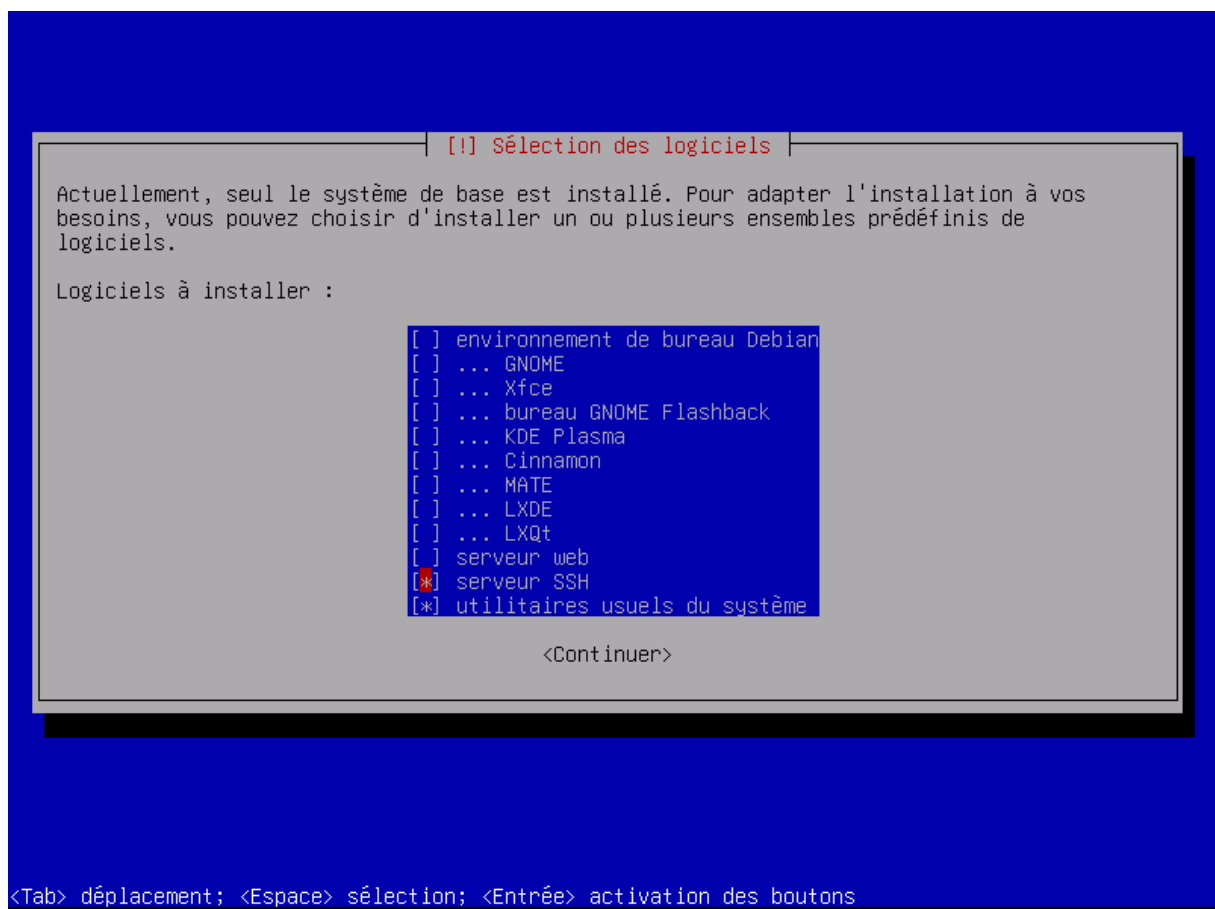


Partitionnement du disque (utilisé un disque entier et ensuite un partitionnement assisté)

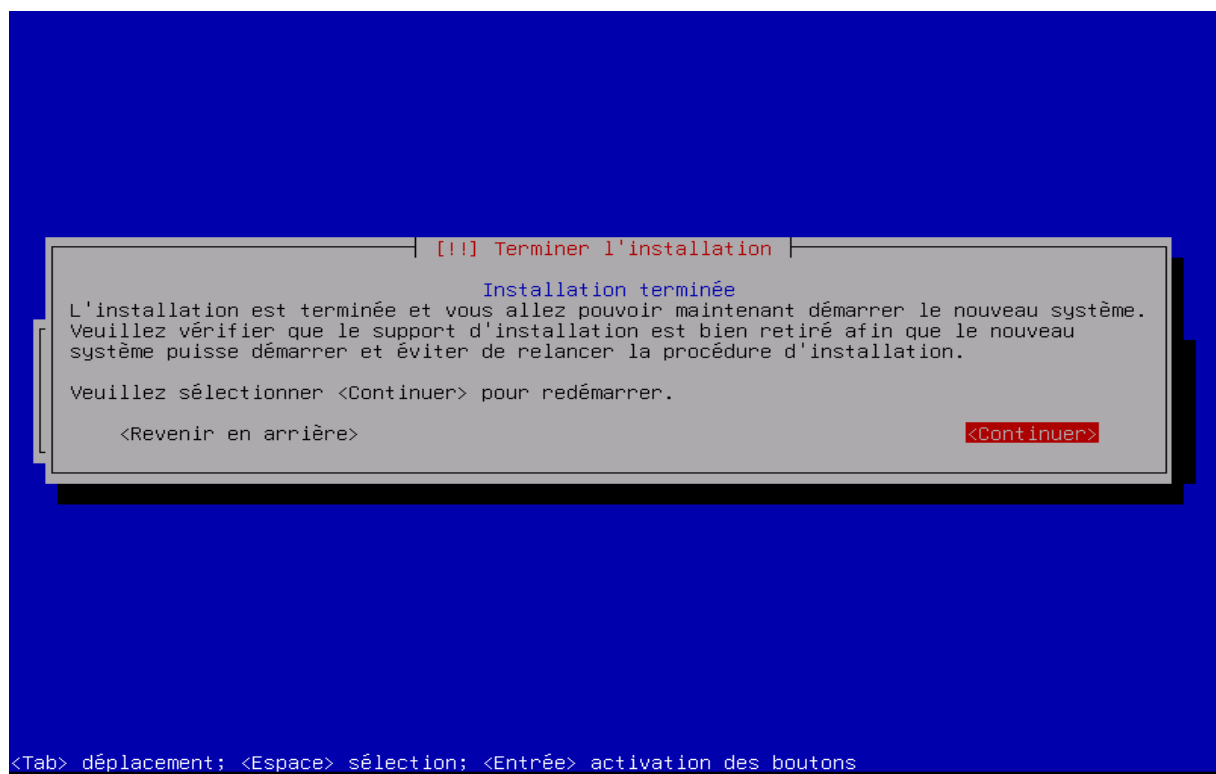
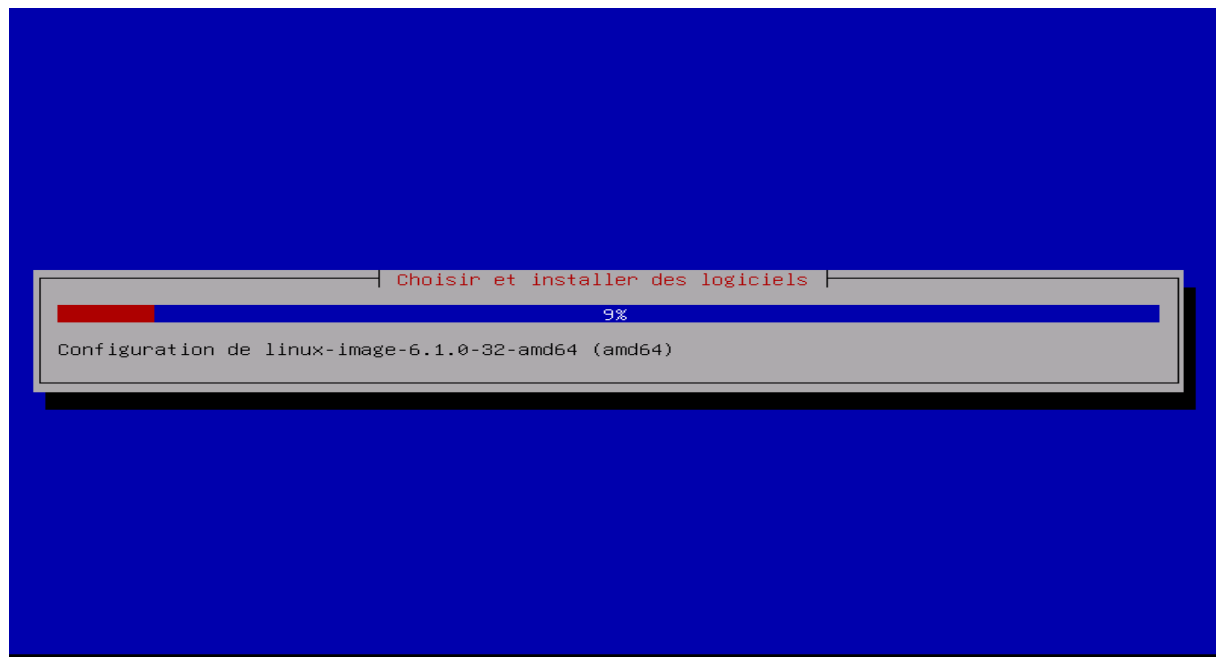




Sélectionner que le serveur ssh et les utilitaires usuels du système pour avoir un système minimaliste sans interface graphique



<Tab> déplacement; <Espace> sélection; <Entrée> activation des boutons



On a machine allumer qui attendent qu'on se connecte pour commencer les configs.

```
Debian GNU/Linux 12 kr-node-001 tty1
kr-node-001 login: _
```

Configuration des hostname

Pour chaque machine : `hostnamectl set-hostname « nom de la machine »` (par exemple : `kr-node-001` pour la première machine)

```
root@kr-node-001:/home/kouadio# hostnamectl set-hostname kr-node-001
root@kr-node-001:/home/kouadio# _
```

Vérification du hostname avec : `cat /etc/hostname` (valable pour les deux autres machines)

```
GNU nano 7.2 /etc/hostname
kr-node-001
```

[Lecture de 1 ligne]

␣ Aide ␣ Écrire Chercher Couper Exécuter Emplacement M-U Annuler M-A Marquer M-] -> Crochet M-Q Précédent
␣ Quitter ␣ Lire fich. ␣ Remplacer ␣ Coller Justifier ~ Aller ligne M-E Refaire M-G Copier ␣ Retrouver M-K Suivant

Configuration des adresses ip statiques pour chaque machine :

Kr-node-001 : `sudo nano /etc/network/interfaces` (puis on ajoute l'ip le mask et le gateway)

```
GNU nano 7.2 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens33
iface ens33 inet static
    address 192.168.15.11
    netmask 255.255.255.0
    gateway 192.168.15.2
```

[15 lignes écrites]

␣ Aide ␣ Écrire Chercher Couper Exécuter Emplacement M-U Annuler M-A Marquer M-] -> Crochet M-Q Précédent
␣ Quitter ␣ Lire fich. ␣ Remplacer ␣ Coller Justifier ~ Aller ligne M-E Refaire M-G Copier ␣ Retrouver M-K Suivant

Redémarrer le réseau : sudo systemctl restart networking puis ip a pour vérifier l'ip

```
gateway 192.168.15.2

root@kr-node-001:/home/kouadio# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:33:c0:fe brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.15.11/24 brd 192.168.15.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe33:c0fe/64 scope link
        valid_lft forever preferred_lft forever
root@kr-node-001:/home/kouadio#
```

Kr-node-002 : sudo nano /etc/network/interfaces (puis on ajoute l'ip le mask et le gateway)

```
GNU nano 7.2 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens33
iface ens33 inet static
    address 192.168.15.12
    netmask 255.255.255.0
    gateway 192.168.15.2

[ 15 lignes écrites ]
 Aide  Écrire  Chercher  Couper  Exécuter  Emplacement  Annuler  Marquer  H-T -> Crochet  H-Q Précédent
 Quitter  Lire fich. Remplacer  Coller  Justifier  Aller ligne  Refaire  Copier  Retrouver  Suivant
```

Redémarrer le réseau : sudo systemctl restart networking puis ip a pour vérifier l'ip

```
gateway 192.168.15.2

root@kr-node-002:/home/kouadio# systemctl restart networking
root@kr-node-002:/home/kouadio# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:35:db:17 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.15.12/24 brd 192.168.15.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe35:db17/64 scope link
        valid_lft forever preferred_lft forever
root@kr-node-002:/home/kouadio#
```

kr-node-003 : sudo nano /etc/network/interfaces (puis on ajoute l'ip le mask et le gateway)

```
GNU nano 7.2 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens33
iface ens33 inet static
    address 192.168.15.13
    netmask 255.255.255.0
    gateway 192.168.15.2_

[ 15 lignes écrites ]
G Aide      O Écrire  M Chercher  K Couper   ~ Exécuter  ^C Emplacement M-U Annuler  M-A Marquer  M-T -> Crochet M-Q Précédent
X Quitter   R Lire fich.  ~ Remplacer ~ Coller   ~ Justifier ~ Aller ligne M-E Refaire  M-G Copier   O Retrouver  M-K Suivant
```

Redémarrer le réseau : `sudo systemctl restart networking` puis `ip a` pour vérifier l'ip

```
gateway 192.168.15.2

root@kr-node-003:/home/kouadio# ip a
root@kr-node-003:/home/kouadio# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:5a:77:9a brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.15.13/24 brd 192.168.15.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe5a:779a/64 scope link
        valid_lft forever preferred_lft forever
root@kr-node-003:/home/kouadio#
```

Résolution de noms par fichier statique pour garantir que les machines se voient par leur hostname sans DNS.

Commande : **`sudo nano /etc/hosts`** puis on modifie le fichier comme suit (valable sur les trois machines virtuelles).

```
GNU nano 7.2 /etc/hosts *
127.0.0.1    localhost
127.0.1.1    kr-node-001.it.com    kr-node-001
192.168.15.11 kr-node-001
192.168.15.12 kr-node-002
192.168.15.13 kr-node-003

# The following lines are desirable for IPv6 capable hosts
::1    localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

G Aide      O Écrire  K Chercher  K Couper    ~ Exécuter  C Emplacement M-U Annuler  M-A Marquer  M-] -> Crochet M-Q Précédent
X Quitter   R Lire fich. ~U Remplacer ~U Collier  ~ Justifier ~ Aller ligne M-E Refaire  M-G Copier    O Retrouver  M-K Suivant
```

Redémarrer le réseau : `sudo systemctl restart networking` puis les tests de ping pour vérifier la communication entre les différentes machines virtuelles.

```
root@kr-node-003:/home/kouadio# systemctl restart networking
root@kr-node-003:/home/kouadio# ping kr-node-001
PING kr-node-001 (192.168.15.11) 56(84) bytes of data.
64 bytes from kr-node-001 (192.168.15.11): icmp_seq=1 ttl=64 time=2.03 ms
64 bytes from kr-node-001 (192.168.15.11): icmp_seq=2 ttl=64 time=0.665 ms
64 bytes from kr-node-001 (192.168.15.11): icmp_seq=3 ttl=64 time=2.06 ms
64 bytes from kr-node-001 (192.168.15.11): icmp_seq=4 ttl=64 time=1.12 ms
64 bytes from kr-node-001 (192.168.15.11): icmp_seq=5 ttl=64 time=0.564 ms
64 bytes from kr-node-001 (192.168.15.11): icmp_seq=6 ttl=64 time=0.849 ms
64 bytes from kr-node-001 (192.168.15.11): icmp_seq=7 ttl=64 time=1.34 ms
64 bytes from kr-node-001 (192.168.15.11): icmp_seq=8 ttl=64 time=0.648 ms
^C
--- kr-node-001 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7066ms
rtt min/avg/max/mdev = 0.564/1.160/2.057/0.565 ms
root@kr-node-003:/home/kouadio# ping kr-node-002
PING kr-node-002 (192.168.15.12) 56(84) bytes of data.
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=1 ttl=64 time=1.68 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=2 ttl=64 time=1.84 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=3 ttl=64 time=1.28 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=4 ttl=64 time=0.785 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=5 ttl=64 time=2.08 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=6 ttl=64 time=1.98 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=7 ttl=64 time=0.797 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=8 ttl=64 time=0.748 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=9 ttl=64 time=2.95 ms
64 bytes from kr-node-002 (192.168.15.12): icmp_seq=10 ttl=64 time=0.708 ms
^C
--- kr-node-002 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9072ms
rtt min/avg/max/mdev = 0.708/1.485/2.945/0.710 ms
root@kr-node-003:/home/kouadio#
```

Configuration SSH

Pour permettre les connexions **SSH sans mot de passe** entre mes trois machines :

- Étape 1 : Générer une clé SSH sur chaque machine

Commande : `ssh-keygen -t ed25519`

```
kouadio@kr-node-002:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kouadio/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kouadio/.ssh/id_ed25519
Your public key has been saved in /home/kouadio/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:H0daKvTea10V2kAtGsfIhe3CU/PznupeLlZCsGniS0 kouadio@kr-node-002
The key's randomart image is:
+--[ED25519 256]--+
|      .+o+o+o+   |
|      +.+.+.+.   |
|      o. .+o+    |
|      .+ +.      |
|      S.o8..o    |
|      ..+o.o     |
|      .o+ .o     |
|      E. o+...   |
|      .o+.+     |
+-----[SHA256]-----+
kouadio@kr-node-002:~$
```


- Étape 2 : Copier les clés sur les autres machines

Commende pour la machine kr-node-001 :

ssh-copy-id kouadio@kr-node-002

ssh-copy-id kouadio@kr-node-003

```
kouadio@kr-node-002:~$ ssh-copy-id kouadio@kr-node-001
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kouadio/.ssh/id_ed25519.pub"
The authenticity of host 'kr-node-001 (192.168.15.11)' can't be established.
ED25519 key fingerprint is SHA256:/YbqNHIE20Geg7onHtiKbLSwCt7fztmd6a0JE3i1mmw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
kouadio@kr-node-001's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'kouadio@kr-node-001'"
and check to make sure that only the key(s) you wanted were added.

kouadio@kr-node-002:~$ ssh-copy-id kouadio@kr-node-003
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kouadio/.ssh/id_ed25519.pub"
The authenticity of host 'kr-node-003 (192.168.15.13)' can't be established.
ED25519 key fingerprint is SHA256:/YbqNHIE20Geg7onHtiKbLSwCt7fztmd6a0JE3i1mmw.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
kouadio@kr-node-003's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'kouadio@kr-node-003'"
and check to make sure that only the key(s) you wanted were added.

kouadio@kr-node-002:~$ _
```

1- 2ème partie : Créer un cluster k3s

Mettre à jour les paquets :

- sudo apt update
- sudo apt upgrade -y

Installation de curl

- sudo apt install curl

Installation de k3s uniquement sur la machine controlleur (kr-node-001)

- curl -sL https://get.k3s.io | sh -

```
kouadio@kr-node-001: ~  
kouadio@kr-node-001:~$ su  
Mot de passe :  
root@kr-node-001:/home/kouadio# curl -sfL https://get.k3s.io | sh -  
  
[INFO] Finding release for channel stable  
[INFO] Using v1.32.3+k3s1 as release  
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.32.3  
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.32.3  
[INFO] Verifying binary download  
[INFO] Installing k3s to /usr/local/bin/k3s  
[INFO] Skipping installation of SELinux RPM  
[INFO] Creating /usr/local/bin/kubectl symlink to k3s  
[INFO] Creating /usr/local/bin/crictl symlink to k3s  
[INFO] Creating /usr/local/bin/ctr symlink to k3s  
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh  
[INFO] Creating uninstall script /usr/local/bin/k3s-uninstall.sh  
[INFO] env: Creating environment file /etc/systemd/system/k3s.service.env  
[INFO] systemd: Creating service file /etc/systemd/system/k3s.service  
[INFO] systemd: Enabling k3s unit  
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/s  
[INFO] Host iptables-save/iptables-restore tools not found  
[INFO] Host ip6tables-save/ip6tables-restore tools not found  
[INFO] systemd: Starting k3s  
root@kr-node-001:/home/kouadio#
```

Récupération du Token d'Inscription

Pour permettre aux nœuds workers de rejoindre le cluster, il est nécessaire de récupérer le token d'inscription généré par K3s sur le Contrôleur.

Sur kr-node-001, exécutons : `cat /var/lib/rancher/k3s/server/node-token`

Le token obtenu sera utilisé par chacun des nœuds workers afin d'authentifier leur connexion avec le nœud Contrôleur.

Installation de K3s en mode Worker

Sur chaque nœud worker (kr-node-002 et kr-node-003), nous installons K3s en précisant l'URL du serveur Contrôleur ainsi que le token d'inscription.

Voici la commande utilisée (en remplaçant `<IP_CONTROLLER>` par le nom du Contrôleur (kr-node-001), et `<TOKEN>` par le token récupéré) : `curl -sfL https://get.k3s.io | K3S_URL=https://<IP_CONTROLLER>:6443 K3S_TOKEN=<TOKEN> sh -`

Cette commande permet aux workers de se connecter au cluster en toute sécurité et de rejoindre automatiquement le groupe de nœuds.

```

kouadio@kr-node-003: ~
root@kr-node-001:/home/kouadio# cat /var/lib/rancher/k3s/server/node-token
Kl0c97a20e8ad0460817b700a6d735a938cf18dcc2711a80a4d74d342a31ccea9da:server:9377c68225e6354375a4e67a5ede558e
root@kr-node-001:/home/kouadio# exit
exit
kouadio@kr-node-001:~$ ssh kr-node-003
Linux kr-node-003 6.1.0-32-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.129-1 (2025-03-06) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Apr  9 16:49:20 2025
kouadio@kr-node-003:~$ su
Mot de passe :
root@kr-node-003:/home/kouadio# curl -sfl https://get.k3s.io | K3S_URL=https://kr-node-001:6443 K3S_TOKEN=Kl0c97a20e8ad0460817b700a6d735a938cf18dcc2711a80a4d
74d342a31ccea9da:server:9377c68225e6354375a4e67a5ede558e sh -
[INFO] Finding release for channel stable
[INFO] Using v1.32.3+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.32.3+k3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.32.3+k3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubect1 symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s-agent.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO] systemd: Enabling k3s-agent unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s-agent.service - /etc/systemd/system/k3s-agent.service.
[INFO] Host iptables-save/iptables-restore tools not found
[INFO] Host ip6tables-save/ip6tables-restore tools not found
[INFO] systemd: Starting k3s-agent
root@kr-node-003:/home/kouadio#

```

Vérification du Cluster

Une fois les installations terminées, il est essentiel de vérifier l'état du cluster. Sur le nœud Contrôleur, la commande suivante affiche l'ensemble des nœuds du cluster ainsi que leur IP interne (ce qui permet de valider que la communication interne fonctionne correctement) : `sudo kubect1 get nodes -o wide`

```

kouadio@kr-node-001: ~
root@kr-node-001:/home/kouadio# kubect1 get nodes
NAME             STATUS    ROLES                  AGE     VERSION
kr-node-001      Ready     control-plane,master   33m     v1.32.3+k3s1
kr-node-002      Ready     <none>                  49s     v1.32.3+k3s1
kr-node-003      Ready     <none>                  14m     v1.32.3+k3s1
root@kr-node-001:/home/kouadio#

```

Déploiement de l'Application NGINX

Nous avons utilisé une image NGINX pour effectuer un test simple : `kubect1 create deployment nginx --image=nginx`

Ensuite, nous avons augmenté le nombre de répliques (pods) à 3 pour voir la répartition : `kubect1 scale deployment nginx --replicas=3`

```

kouadio@kr-node-001: ~
root@kr-node-001:/home/kouadio# kubect1 get pods
NAME             READY     STATUS              RESTARTS   AGE
nginx-5869d778c-5p454  0/1      ImagePullBackOff    0           2m4s
root@kr-node-001:/home/kouadio# kubect1 scale deployment nginx --replicas=3
deployment.apps/nginx scaled
root@kr-node-001:/home/kouadio# kubect1 get pods -o wide
NAME             READY     STATUS              RESTARTS   AGE     IP             NODE             NOMINATED NODE   READINESS GATES
nginx-5869d778c-2qdfz  0/1      ErrImagePull        0           12s     10.42.0.14     kr-node-001      <none>            <none>
nginx-5869d778c-5p454  0/1      ImagePullBackOff    0          2m30s    10.42.1.4      kr-node-003      <none>            <none>
nginx-5869d778c-lr52r  0/1      ErrImagePull        0           12s     10.42.2.4      kr-node-002      <none>            <none>
root@kr-node-001:/home/kouadio#

```

Le cluster K3s est bien fonctionnel et opérationnel : tous les nœuds sont actifs, communiquent entre eux, et le plan de contrôle répond correctement.

👉 Le problème rencontré ne vient pas du cluster lui-même, mais de l'impossibilité des nœuds de télécharger les images depuis Internet, liée à la configuration réseau actuelle des machines virtuelles.